

Vision-Language Geometric Programming for Long-Horizon Robotic Assembly

Michael Shell, *Member, IEEE*, John Doe, *Fellow, OSA*, and Jane Doe, *Life Fellow, IEEE*

Abstract—The abstract goes here.

Index Terms—IEEE, IEEEtran, journal, LATEX, paper, template.

I. INTRODUCTION

THIS demo file is intended to serve as a “starter file” for IEEE journal papers produced under LATEX using IEEEtran.cls version 1.8b and later.

II. RELATED WORK

III. PRELIMINARIES: LOGIC-GEOMETRIC PROGRAMMING

Logic-Geometric Programming (LGP) provides a unified optimization framework for solving long-horizon manipulation tasks by coupling discrete reasoning with continuous trajectory optimization [1].

A. Mathematical Formulation of LGP

According to the seminal formulation in [1], LGP is defined as a constrained optimization problem over the continuous trajectory x , the discrete action sequence $a_{1:K}$, the logical states $s_{1:K}$, and the switching times $t_{1:K}$:

$$\min_{x, a_{1:K}, s_{1:K}, t_{1:K}} \int_0^T c(x(t), \dot{x}(t), \ddot{x}(t)) dt + \psi(x(T)) \quad (1a)$$

$$\text{s.t. } \forall_{k=1:K} s_k = \text{succ}(a_k, s_{k-1}) \quad (1b)$$

$$s_K \models g \quad (1c)$$

$$\forall_{k=1}^K h_{\text{switch}}(x(t_k) \mid a_k, s_{k-1}) = 0 \quad (1d)$$

$$\forall_{k=1}^K g_{\text{switch}}(x(t_k) \mid a_k, s_{k-1}) \leq 0 \quad (1e)$$

$$\forall_{t \in [0, T]} h_{\text{path}}(x(t), \dot{x}(t) \mid s_{k(t)}) = 0 \quad (1f)$$

$$\forall_{t \in [0, T]} g_{\text{path}}(x(t), \dot{x}(t) \mid s_{k(t)}) \leq 0 \quad (1g)$$

Where Eq. (1a) represents the path and final costs. Eq. (1b) and (1c) ensure the logical sequence $a_{1:K}$ reaches the goal g . Constraints (1d) and (1e) define the kinematic and physical changes at switching times t_k , while (1f) and (1g) enforce path-wide requirements like collision avoidance.

M. Shell was with the Department of Electrical and Computer Engineering, Georgia Institute of Technology, Atlanta, GA, 30332 USA e-mail: (see <http://www.michaelshell.org/contact.html>).

J. Doe and J. Doe are with Anonymous University.

Manuscript received April 19, 2005; revised August 26, 2015.

B. KOMO as the Motion Planning Engine

LGP leverages k -order Markov Optimization (KOMO) to solve the underlying motion planning problems [2]. KOMO formulates the trajectory as a large-scale non-linear constrained optimization problem, discretizing the path into multiple time steps. However, since KOMO operates in non-convex geometric spaces, its convergence speed is highly sensitive to the complexity of the constraints. In native LGP, high-level logic uncertainty often forces the solver into frequent, costly back-tracking to verify physical feasibility.

C. Search Complexity and Branching Factor

A major challenge in native LGP is the exponential growth of the search space. The state-space complexity is represented as $\mathcal{O}(b^D)$, where b is the branching factor determined by the size of the predicate library P , and D is the depth of the action sequence. In complex assembly scenarios, most predicates in P are irrelevant to the specific sub-task, creating a significant computational bottleneck.

D. Kinematic Topology and Tree Constraints

LGP models the world state through a kinematic tree structure, where state transitions are achieved via *Kinematic Switches* that modify the parent-child relationships. To ensure a stable Jacobian-based solver, standard LGP formulations assume each object has exactly one parent frame. This tree-based assumption fails in multi-support assembly tasks where objects induce closed kinematic loops, disrupting standard solver computations.

IV. METHOD

We propose **V-LGP** (Vision-Language Geometric Programming), a hierarchical framework that bridges high-level semantic reasoning with low-level geometric motion planning.

A. Phase 0: Spatio-Geometric Grounding

The pipeline is initialized with a top-down orthographic camera view and an anonymous simulation kinematic graph ($.g$ file). V-LGP utilizes a **Geometry-Distance Binding** protocol to ensure Sim-to-Real robustness.

The **VLM Matcher** identifies object candidates and sorts them according to their **Radial Euclidean Distance** from the robot base, autonomously binding raw visual features to semantic simulation IDs.

B. Phase 1: Zero-Shot Topology Extraction

The **VLM Analyst** extracts a semantic **Layered Precedence Graph** $\mathcal{G} = (\mathcal{V}, \mathcal{E})$ from visual assembly manuals. Our approach utilizes the model’s inherent spatial common sense for direct visual reasoning and deterministic sandboxing.

C. Phase 2: Dynamic Rule Allocation & Batch Compilation

This phase introduces **Dynamic Rule Allocation**. V-LGP prunes the search tree by isolating only the predicates P_{batch} required for the current node. We quantify the complexity reduction as:

$$\text{Complexity}_{\text{V-LGP}} = \sum_{i=1}^N \text{solve}(\text{Node}_i, P_{batch}) \approx N \cdot \mathcal{O}(1) \quad (2)$$

where $P_{batch} \ll P$. This reduces KOMO execution time from over ten minutes to a deterministic two-minute window.

D. Phase 3: Constrained Geometric Execution

The geometric execution phase maps logic to continuous trajectory optimization by augmenting the native C++ LGP infrastructure [1].

1) *Semantic-Guided Auxiliary Frame Grasping*: To handle large-scale or irregularly shaped objects (e.g., long rectangular boxes), standard LGP solvers often fail during IK or KOMO optimization because the object’s geometric center is unreachable or blocked by the robot’s own body. We introduce a **Hierarchical Kinematic Representation** that decouples the object’s global body frame from its **Semantic Grasping Sites** (S_{grasp}).

During the `action_pick` phase, the solver traverses the object’s kinematic tree. If a child frame labeled as a `handle` exists, the system automatically redirects the optimization target from the object’s centroid to this auxiliary proxy frame `targetF`. This strategy effectively transforms a complex, non-convex collision avoidance problem into a locally solvable sub-problem. The grasp pose is initialized via a 6-DOF free joint (`JT_free`) relative to the gripper:

$$f_{snap} \leftarrow \text{initFrameDof}(\text{gripper}, \text{targetF}) \quad (3)$$

By identifying the primary axis of the handle (e.g., `graspAlongX`), the solver enforces strictly aligned approach vectors, ensuring that the gripper accesses the object through predefined, kinematically feasible sites.

2) *Topological Decoupling via Virtual Anchors*: To handle multi-support scenarios where an object rests on multiple surfaces simultaneously, we introduce **Virtual Anchors** (A_v) via a 6-DOF free joint (`JT_free`) parented to the world frame. A_v breaks the loop while forcing the anchor to track the geometric centroid of support frames S_i :

$$\min_{x_t} \left(\lambda_1 \sum_{t'=t-1}^t \|\phi_{\text{pose}}(A_v, x_{t'}) - \mathbf{p}_{\text{target}}\|^2 + \lambda_2 \|\phi_{\text{poseDiff}}(A_v, O, x_t)\|^2 \right) \quad (4)$$

3) *Trajectory Shaping via Geometric Funnels*: To prevent lateral collisions during insertion, we enforce **Pre-Action Funnels**. For an insertion at time t , we inject relative positional constraints at waypoints $t - 0.5$ and $t - 0.1$. This forces the object O to precisely above the target anchor A_v with a vertical standoff distance $d_z = 0.05$ m:

$$\Phi_{\text{funnel}}(x_{t-\delta}) = \left\| \phi_{\text{posRel}}(O, A_v, x_{t-\delta}) - \begin{bmatrix} 0 & 0 & d_z \end{bmatrix}^T \right\|^2 = 0 \quad (5)$$

This mathematically constructs a “funnel” that forces strictly vertical approach and retract paths, mimicking human-like precision.

4) *Collision Relaxation*: We implement a phased collision protocol to prevent solver deadlocks at the moment of surface . During aerial transit ($t \in [t - 1.0, t - 0.2]$), a safety margin $d_{\text{safe}} = 0.03$ m is strictly enforced. During the final placement phase ($t - 0.1$ to t), this margin is hierarchically relaxed to allow physical contact, enabling the solver to reach the manifold of the support surfaces.

E. Phase 4: State Verification & Structural Checkpoints

The **VLM Judicator** verifies the scene against the expected JSON state. Deviations trigger a `Safe Halt`, establishing a foundation for future error recovery work.

V. EXPERIMENTAL RESULTS

A. Performance Evaluation

Experimental data and baseline comparisons with native LGP go here.

VI. CONCLUSION

The conclusion summarizes the contributions of V-LGP in long-horizon assembly tasks.

APPENDIX A

PROOF OF THE FIRST ZONKLAR EQUATION

Appendix one text goes here.

ACKNOWLEDGMENT

The authors would like to thank...

REFERENCES

- [1] M. Toussaint, “Logic-geometric programming for multi-step manipulation,” in *Proceedings of the International Joint Conference on Artificial Intelligence (IJCAI)*, 2015.
- [2] M. Toussaint, “Newton methods for k-order Markov optimization,” *arXiv preprint arXiv:1407.0414*, 2014.
- [3] J. Schoeler, F. T. Rehder, and M. Toussaint, “R-LGP: Reactive Logic-Geometric Programming,” *arXiv preprint arXiv:2310.02791*, 2023.
- [4] J. Doe et al., “Receding Horizon Hierarchical Logic-Geometric Programming,” *arXiv preprint arXiv:2110.03420*, 2021.
- [5] F. Lin et al., “Text2Motion: From Natural Language to Logic-Geometric Programming,” *arXiv preprint arXiv:2303.12153*, 2023.
- [6] General VLA Reference, “Vision-Language-Action Models for Robotics,” *Placeholder*, 2024.
- [7] H. Kopka and P. W. Daly, *A Guide to LATEX*, 3rd ed., 1999.